# CryptoVault

*Crypto-Agile Identity, Authentication & Secure Data Platform*

Full Implementation Plan — MySQL Edition

---

## 1. Executive Summary

### 1.1 What It Does

CryptoVault is a banking-inspired security platform that provides authentication, authorization, encrypted storage, cryptographic key management, algorithm migration, audit logging, and a path toward post-quantum readiness. Concretely, it lets an organization:

- Register and authenticate users with secure password handling and JWTs
- Manage role-based permissions (admin vs. standard user)
- Store sensitive information encrypted at rest, never in plaintext
- Rotate cryptographic keys without downtime
- Migrate between encryption algorithms via configuration, not code rewrites
- Maintain a complete, queryable audit trail of security-relevant actions

### 1.2 Target Users

**Primary**

- FinTech startups
- Banks
- Healthcare systems
- Government systems

**Secondary**

- Developers learning security engineering
- Security researchers

### 1.3 Core Value Proposition

Most systems hardcode their cryptographic algorithm directly into application logic. When that algorithm becomes weak, deprecated, or fails a compliance audit, the typical remediation path looks like this:

```
Application
  ↓
Rewrite Code
  ↓
Retest
  ↓
Redeploy
```

This is slow, risky, and expensive — every call site that touches encryption has to be found, changed, and re-verified. CryptoVault's crypto-agility layer flips this:

```
Application
  ↓
Change Config
  ↓
Migration Job
  ↓
Done
```

The application code never references a specific algorithm by name. It calls an abstract “encrypt” or “decrypt” operation, and a configuration value (or database flag) determines which concrete algorithm actually runs underneath. Old data keeps a record of which algorithm and key version encrypted it, so nothing breaks when the default changes.

### 1.4 Why People Would Use It

- Future-proof security posture as algorithms age out
- Lower-friction crypto migrations (config change vs. full rewrite)
- Easier regulatory compliance (PCI-DSS, HIPAA, RBI/financial regulations often mandate algorithm agility or timely deprecation)
- Reduced engineering effort per migration cycle
- Stronger auditability for security reviews and incident response

## 2. Market Analysis

### 2.1 Existing Competitors

#### Auth0

Strengths:

- Enterprise authentication
- MFA
- SSO

Weaknesses:

- Not focused on crypto-agility
- Expensive at scale

### Okta

Strengths:

- Identity management

Weaknesses:

- Vendor lock-in
- Limited crypto experimentation

### HashiCorp Vault

Strengths:

- Secrets management
- Enterprise-grade

Weaknesses:

- Complex setup
- No educational transparency

## 2.2 Differentiation

CryptoVault's focus areas, in order of priority:

1. Crypto agility (the core differentiator)
2. Key versioning
3. Algorithm migration tooling
4. Post-quantum readiness path
5. A visual security dashboard for transparency

# 3. Feature Planning

## 3.1 MVP

### Authentication

- Register
- Login
- Logout

### Security

- BCrypt password hashing
- JWT authentication

### Vault

- Store encrypted records

### Roles

- Admin

- User

### Audit

- Login logs
- Vault access logs

## 3.2 V1

### Crypto Agility Layer

Algorithms:

- AES-256-GCM
- ChaCha20-Poly1305

Configuration switches the active algorithm without code changes:

```
algorithm: AES
# or
algorithm: CHACHA20
```

### Key Rotation

Key versions are tracked explicitly:

- v1
- v2
- v3

### Admin Dashboard

- Active users
- Algorithm statistics
- Audit trail

## 3.3 V2

### MFA

- TOTP
- Email OTP

### Security Analytics

- Failed login monitoring
- Suspicious access alerts

### Re-encryption Jobs

Background migration moves existing records from one algorithm to another without downtime:

```
AES
  ↓
ChaCha20
(background migration)
```

### 3.4 V3

**Post-Quantum**

Algorithms:

- ML-KEM (key encapsulation)
- ML-DSA (digital signatures)

Hybrid mode (classical + post-quantum, recommended during the transition period):

```
X25519 + ML-KEM
```

# 4. User Flow

## 4.1 Registration

```
User
  ↓
Register
  ↓
Password Hash (bcrypt)
  ↓
MySQL Database
  ↓
JWT Issued
```

## 4.2 Login

```
Login
  ↓
Password Verification
  ↓
MFA (if enabled)
  ↓
JWT Issued
  ↓
Dashboard
```

## 4.3 Vault Storage

```
Data Input
  ↓
Encryption Service
  ↓
Encrypted Blob
  ↓
MySQL Database
```

## 4.4 Retrieval

```
Encrypted Data
  ↓
Detect Algorithm + Key Version
  ↓
Decrypt
  ↓
Display
```

### 4.5 Edge Cases to Handle Explicitly

- Wrong password (rate-limit and log attempts; never reveal whether the email exists)
- Expired token (return 401 and force re-login; refresh-token flow handles this gracefully)
- Disabled user (block at the authentication layer, before password check or after — decide consistently)
- Rotated key (retrieval must still locate the correct historical key by key_version)
- Deleted algorithm (never hard-delete an algorithm definition while any record still references it; mark as 'retired' instead)
- Migration interruption (re-encryption jobs must be resumable/idempotent, tracked row-by-row)

## 5. System Architecture

High-level component diagram:

```
React Frontend
      |
      v
FastAPI Backend
      |
  +------------------+
  | Authentication   |
  | Authorization    |
  | Audit Service    |
  | Vault Service    |
  | Crypto Engine    |
  | Key Manager      |
  +------------------+
      |
      v
   MySQL  <----->  Redis Cache
```

Notes on this architecture with MySQL specifically:

- MySQL is the single source of truth for users, roles, vault records, key metadata, and audit logs.
- Redis sits alongside MySQL for fast-changing, expendable data: JWT blacklists, rate-limit counters, OTP codes, and session caches. Nothing in Redis should be the only copy of anything important.
- All five core services (Auth, Authorization, Audit, Vault, Crypto Engine, Key Manager) run inside the same FastAPI app for the MVP. Splitting them into separate microservices is a later-stage decision, not a day-one one.

## 6. Recommended Tech Stack (MySQL Edition)

### 6.1 Frontend

React + TypeScript

Why:

- Industry standard
- Strong resume value

Alternative: Next.js

## 6.2 Backend

FastAPI

Why:

- Python ecosystem
- Easy JWT integration
- Native async support, which matters once you add background re-encryption jobs

Alternative: Spring Boot — if targeting banks specifically, Spring Boot is actually the stronger signal, since it's what most banking back-office systems run on. Spring Boot's MySQL story is mature (Spring Data JPA + Hibernate dialect support is excellent).

## 6.3 Database — MySQL

You've chosen MySQL over the originally suggested PostgreSQL. Here's how that changes the plan, concretely:

| Concern | PostgreSQL (original plan) | MySQL (this plan) |
|---|---|---|
| Primary key type | Native UUID type | CHAR(36) or BINARY(16) — MySQL 8.0 has no native UUID column type |
| JSON support | JSONB (indexed, binary) | JSON type (MySQL 8.0+) — functional but less mature indexing than JSONB |
| Transactions/ACID | Full ACID by default | Full ACID, but only with the InnoDB engine — must explicitly avoid MyISAM |
| Enum-like fields (roles, algorithm status) | Often a CHECK constraint or native ENUM | Native ENUM column type, or a lookup table (lookup table is more flexible long-term) |
| Migration tool | Alembic | Alembic still works identically via SQLAlchemy + PyMySQL/mysqlclient driver |
| Case sensitivity | Case-sensitive by default | Depends on collation — must set utf8mb4_bin or similar for email lookups if case-sensitivity matters |

Concrete MySQL setup decisions for CryptoVault:

- Use MySQL 8.0+ specifically — it adds window functions, CTEs, native JSON, and better UTF-8 (utf8mb4) handling, all of which the audit/analytics features will want.
- Force the InnoDB storage engine on every table. InnoDB is the default in modern MySQL, but verify it explicitly in migrations — MyISAM does not support foreign keys or transactions, both of which this schema relies on.
- Use charset utf8mb4 with collation utf8mb4_0900_ai_ci (or utf8mb4_bin if you want case-sensitive email/username comparisons) at the database, table, and column level.
- Store UUIDs as CHAR(36) for readability during development, or BINARY(16) for storage efficiency once you're optimizing. CHAR(36) is the pragmatic MVP choice — switch later if index size becomes a real bottleneck.
- Use SQLAlchemy with the PyMySQL driver (pure Python, simplest install) for development, and consider mysqlclient (C-extension, faster) for production.

Alternative: PostgreSQL remains a fine choice if you ever revisit this decision — it's a closer fit for JSONB-heavy audit querying — but everything below is planned around MySQL as specified.

### 6.4 Cache — Redis

Why:

- JWT/session token blacklist
- Rate limiting counters
- OTP code storage for MFA (V2)

### 6.5 Crypto

Python cryptography library (provides AES-GCM and ChaCha20-Poly1305 primitives directly).

Later: Open Quantum Safe (liboqs / oqs-python) for the V3 post-quantum layer.

## 7. Database Design — MySQL Schema

Below is the schema translated into concrete MySQL DDL. All tables use InnoDB and utf8mb4.

### 7.1 roles

```
CREATE TABLE roles (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(50) NOT NULL UNIQUE,
  description VARCHAR(255),
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

### 7.2 users

```
CREATE TABLE users (
  id CHAR(36) PRIMARY KEY DEFAULT (UUID()),
  email VARCHAR(255) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  role_id INT NOT NULL,
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
```

```
  FOREIGN KEY (role_id) REFERENCES roles(id),
  INDEX idx_users_email (email)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Note: MySQL 8.0's UUID() generates a value at insert time; alternatively generate the UUID in the application layer (Python's uuid4()) and pass it explicitly — this is the more portable, more testable approach and is what's recommended here.

## 7.3 crypto_keys

```
CREATE TABLE crypto_keys (
  id INT AUTO_INCREMENT PRIMARY KEY,
  version INT NOT NULL UNIQUE,
  algorithm VARCHAR(50) NOT NULL,
  status ENUM('active','retired','revoked') NOT NULL DEFAULT 'active',
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_keys_version (version)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

The raw key material itself should never sit in this table in plaintext. Store it encrypted-at-rest under a master key (e.g. via an environment-provided KMS key, or at minimum an application-level master secret from env vars), or reference an external secret in a vault/KMS service. This table tracks metadata and the key version/algorithm linkage, not raw secrets.

## 7.4 vault_records

```
CREATE TABLE vault_records (
  id CHAR(36) PRIMARY KEY,
  user_id CHAR(36) NOT NULL,
  encrypted_data MEDIUMBLOB NOT NULL,
  algorithm_used VARCHAR(50) NOT NULL,
  key_version INT NOT NULL,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id),
  FOREIGN KEY (key_version) REFERENCES crypto_keys(version),
  INDEX idx_vault_user (user_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

MEDIUMBLOB (up to 16MB) is used instead of BLOB (64KB max) since encrypted records — especially documents or larger payloads — can easily exceed 64KB. Use BLOB only if you're certain every record stays small.

## 7.5 audit_logs

```
CREATE TABLE audit_logs (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id CHAR(36),
  action VARCHAR(100) NOT NULL,
  ip_address VARCHAR(45),
  timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id),
  INDEX idx_audit_user (user_id),
  INDEX idx_audit_timestamp (timestamp)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

ip_address is VARCHAR(45) rather than a numeric type, since this comfortably fits both IPv4 and IPv6 textual representations. user_id is nullable here deliberately — failed login attempts

against a non-existent email still need to be logged for security monitoring, and won't have a valid user_id to attach to.

BIGINT is used for audit_logs.id specifically because this table grows the fastest of all five and will likely need partitioning or archival once it reaches tens of millions of rows; BIGINT avoids an awkward auto-increment ceiling that INT could eventually hit.

### 7.6 Index Summary

| Table | Index | Purpose |
|---|---|---|
| users | idx_users_email | Fast lookup on login (email is also UNIQUE, which MySQL already indexes) |
| audit_logs | idx_audit_user | Filter a user's full audit history quickly |
| audit_logs | idx_audit_timestamp | Time-range queries for the admin dashboard charts |
| vault_records | idx_vault_user | List a user's stored records efficiently |
| crypto_keys | idx_keys_version | Resolve key_version to algorithm/status during decryption |

## 8. Important APIs

### 8.1 Auth

| Method | Endpoint | Description |
|---|---|---|
| POST | /api/auth/register | Create a new user, hash password, issue JWT |
| POST | /api/auth/login | Verify credentials (+ MFA in V2), issue JWT |
| POST | /api/auth/logout | Blacklist current JWT in Redis |

### 8.2 Vault

| Method | Endpoint | Description |
|---|---|---|
| POST | /api/vault/store | Encrypt and store a record under the active algorithm/key |
| GET | /api/vault/{id} | Retrieve and decrypt a record using its stored algorithm/key_version |
| DELETE | /api/vault/{id} | Remove a record (soft-delete recommended for audit integrity) |

### 8.3 Admin

| Method | Endpoint | Description |
|---|---|---|
| GET | /api/admin/audit | Paginated, filterable audit log view |
| POST | /api/admin/rotate-key | Generate a new key version, mark prior as retired |
| POST | /api/admin/migrate-algorithm | Kick off a background re-encryption job (V2+) |

# 9. UI Pages

## 9.1 Landing Page

- What crypto agility is (plain-language explainer)
- Architecture diagram
- Feature highlights

## 9.2 Login Page

Clean, banking-style UI — minimal color palette, clear error states, no playful copy.

## 9.3 Dashboard

Cards:

- Stored Records (count)
- Active Algorithm
- Current Key Version
- Recent Activity feed

## 9.4 Vault Page

Table columns:

- Record Name
- Algorithm
- Version
- Created
- Actions (view / delete)

## 9.5 Admin Panel

Charts:

- Login attempts over time
- Encryption algorithm usage breakdown
- Key rotation history

# 10. Development Roadmap

## Phase 1 — Authentication

- Register
- Login
- JWT issuance/validation
- Role-based access control (RBAC)

## Phase 2 — Vault

- Encryption service wiring
- Storage endpoint
- Retrieval + decryption endpoint

## Phase 3 — Crypto Agility

- AES-256-GCM implementation
- ChaCha20-Poly1305 implementation
- Config-driven algorithm switching

## Phase 4 — Key Rotation

- Key versioning table and logic
- Migration/re-encryption jobs
- Audit dashboard

## Phase 5 — Post-Quantum Layer

- ML-KEM integration
- ML-DSA integration
- Hybrid key exchange (X25519 + ML-KEM)

# 11. Timeline

Estimated effort, assuming consistent, focused work:

| Phase | Duration |
|---|---|
| Phase 1 — Authentication | 2 weeks |
| Phase 2 — Vault | 2 weeks |
| Phase 3 — Crypto Agility | 2 weeks |
| Phase 4 — Key Rotation | 2 weeks |
| Phase 5 — Post-Quantum | 3–4 weeks |

| Phase | Duration |
|---|---|
| Total | 10–12 weeks |

This estimate assumes working consistently and prior familiarity with most of the stack. If this runs alongside a full-time job, coursework, or other major commitments, a realistic planning assumption is to roughly double these durations rather than compress the work into evenings only — underestimating this is the most common reason side projects like this stall around Phase 3.

## 12. Resume Impact

This project demonstrates concrete, verifiable skills across several domains:

**Security**

- Authentication
- Authorization
- JWT
- BCrypt

**Backend**

- REST API design
- FastAPI / Spring Boot

**Databases**

- MySQL
- Schema design
- Indexing strategy

**Architecture**

- Service design
- Config-driven systems

**Cryptography**

- AES
- ChaCha20
- Key rotation
- Post-quantum readiness

## 13. Companies That Would Find This Relevant

- JPMorgan Chase
- Goldman Sachs
- Morgan Stanley

- Barclays
- Deutsche Bank
- Razorpay
- PhonePe

# 14. Suggested Resume Bullet

> *Built CryptoVault, a crypto-agile authentication and secure data platform using FastAPI, MySQL, JWT, and AES/ChaCha20 encryption, supporting key rotation, encrypted storage, role-based access control, audit logging, and post-quantum-ready cryptographic migration architecture.*

# 15. Practical Implementation Checklist (Build Order)

A condensed, step-by-step build order that turns the plan above into an actual sequence of work sessions.

## Step 1 — Environment & Skeleton

- Set up FastAPI project structure (app/, models/, schemas/, services/, routers/)
- Run MySQL 8.0 and Redis via Docker Compose — do not install MySQL natively for development
- Configure SQLAlchemy with PyMySQL driver, set InnoDB + utf8mb4 in connection options
- Set up Alembic for migrations, pointed at the MySQL connection string
- Confirm a /health endpoint returns 200 before writing any business logic

## Step 2 — Database Schema

- Implement all five tables as SQLAlchemy models exactly per Section 7
- Generate and run the first Alembic migration
- Add all indexes from Section 7.6 now, not retroactively
- Seed the roles table with 'admin' and 'user' rows

## Step 3 — Authentication Core

- Register: bcrypt-hash password, insert user row, issue JWT
- Login: verify password, issue JWT
- Logout: blacklist token JTI in Redis with TTL matching token expiry

## Step 4 — RBAC

- Add a FastAPI dependency that decodes the JWT and checks role_id against required role
- Keep it to two roles (admin, user) for MVP — resist adding a permissions matrix early

## Step 5 — Crypto Engine

- Define a CipherStrategy interface: encrypt(plaintext, key) and decrypt(ciphertext, key)
- Implement AES-256-GCM and ChaCha20-Poly1305 concretely using the Python cryptography library
- Algorithm choice comes from config/DB, never from inline conditionals scattered through the codebase

## Step 6 — Vault Service

- Wire the crypto engine into store/retrieve endpoints
- On retrieval, read algorithm_used and key_version from the row to select the correct decryptor
- Test: switch the active algorithm in config, confirm new writes use it while old rows still decrypt correctly

## Step 7 — Key Management & Rotation

- Implement key versioning with status (active/retired/revoked)
- Endpoint to generate a new key version and mark the prior one retired
- New vault writes use the active key; existing rows keep referencing their original version

## Step 8 — Audit Logging

- Add middleware/decorator logging user_id, action, ip_address, timestamp on every auth and vault action
- Verify nullable user_id works correctly for failed logins against unknown emails

## Step 9 — Admin Dashboard & Frontend

- Build React frontend once the API surface is stable: login, dashboard, vault table, admin panel with charts
- Build UI last — building against a moving API wastes time

## Step 10 — Reassess Before V2/V3

- MFA, background re-encryption jobs, and post-quantum crypto are real scope increases, not polish
- Treat Phase 5 (post-quantum) as a stretch goal rather than a committed deliverable given the 10–12 week estimate
- liboqs/oqs-python tooling is less mature than mainstream crypto libraries — budget extra time for documentation gaps